Design Task

Name: Pravin Ganesh Jambulingam
Register number: 192311340

Subject:	Compiler Design / Language Processors	**Stack:**	Lex (FLex) & Yacc (Bison), C / GCC
Total Questions:	5 Parser Implementations	**Requirements:**	Lexer, Parser, Diagrams, Validation, Error Handling

Grammar:

- `S → for(A;C;I) S | if(E) S | id=id;`
- `A → id=num | C → id<num | I → id++ | E → id>id`

Sample Input: `for(i=0;i<5;i++) if(a>b) x=y;`

1. Parser Architecture Flow Diagram

2. Code Implementation

FILE: LEXER1.L

```
#include "y.tab.h"
#include <stdio.h>
#include <stdlib.h>
%}

%option noyywrap

%%

"for"                   { return FOR; }
"if"                    { return IF; }
"<"                     { return LT; }
">"                     { return GT; }
"="                     { return ASSIGN; }
"++"                    { return INC; }
"("                     { return LPAREN; }
")"                     { return RPAREN; }
";"                     { return SEMI; }
[0-9]+                  { return NUM; }
[a-zA-Z_][a-zA-Z0-9_]*  { return ID; }
[
]+              { /* ignore whitespace */ }
.                       { return yytext[0]; }

%%
```

FILE: PARSER1.Y

```
%{
#include <stdio.h>
#include <stdlib.h>

extern int yylex(void);
void yyerror(const char *s);
%}

%token FOR IF LT GT ASSIGN INC LPAREN RPAREN SEMI NUM ID

%%

Start : S { printf("Accepted
"); return 0; } ;

S   : FOR LPAREN A SEMI C SEMI I RPAREN S
    | IF LPAREN E RPAREN S
    | ID ASSIGN ID SEMI
    ;

A   : ID ASSIGN NUM ;
C   : ID LT NUM ;
I   :  ID  INC  ;
E   : ID GT ID ;

%%

void yyerror(const char *s)
    { printf("Rejected
");
}

int main(void) {
    if (yyparse() == 0) return 0;
    return 1;
}
```

3. Output Verification

```
$ yacc -d parser1.y && lex lexer1.l && gcc y.tab.c lex.yy.c -o parser1
$ ./parser1 < input1.txt
```

Result: Accepted

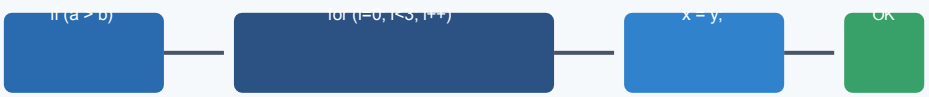
QUESTION 2: Parser for Nested if–for Statements

Grammar:

- S → if(E) S | for(A;C;I) S | id=id;
- E → id>id | A → id=num | C → id<num | I → id++

Sample Input: `if(a>b) for(i=0;i<3;i++) x=y;`

1. Syntax Flow Diagram



2. Code Implementation

FILE: LEXER2.L

```
%{
#include "y.tab.h"
#include <stdio.h>
#include <stdlib.h>
%}

%option noyywrap

%%

"if"                    {  retur   IF; }
                             n

"for"                   {  retur   FOR; }
                             n

">"                     {  retur   GT; }
                             n

"<"                     {  retur   LT; }
                             n

"="                     {  retur   ASSIGN;
                             n     }

"++"                    {  retur   INC; }
                             n

"("                     {  retur   LPAREN;
                             n     }

")"                     {  retur   RPAREN;
                             n     }

";"                     {  retur   SEMI; }
                             n

[0-9]+                  {  retur   NUM; }
                             n

[a-zA-Z_][a-zA-          {  retur   ID; }
Z0-9_]*                      n
[

]+                  { /* ignore whitespace */ }
.                       { return yytext[0]; }

%%
```

FILE: PARSER2.Y

```
%{
#include <stdio.h>
#include <stdlib.h>

extern int yylex(void);
void yyerror(const char *s);
%}

%token IF FOR GT LT ASSIGN INC LPAREN RPAREN SEMI NUM ID

%%

Start : S { printf("Accepted
"); return 0; } ;

S   : IF LPAREN E RPAREN S
    | FOR LPAREN A SEMI C SEMI I RPAREN S
    | ID ASSIGN ID SEMI
    ;

E   : ID GT ID ;
A   : ID ASSIGN NUM ;
C   : ID LT NUM ;
I   : ID INC ;

%%

void yyerror(const char *s)
    { printf("Rejected
");
}

int main(void) {
    if (yyparse() == 0) return 0;
    return 1;
}
```

3. Output Verification

```
$ yacc -d parser2.y && lex lexer2.l && gcc y.tab.c lex.yy.c -o parser2
$ ./parser2 < input2.txt
```

Result: Accepted

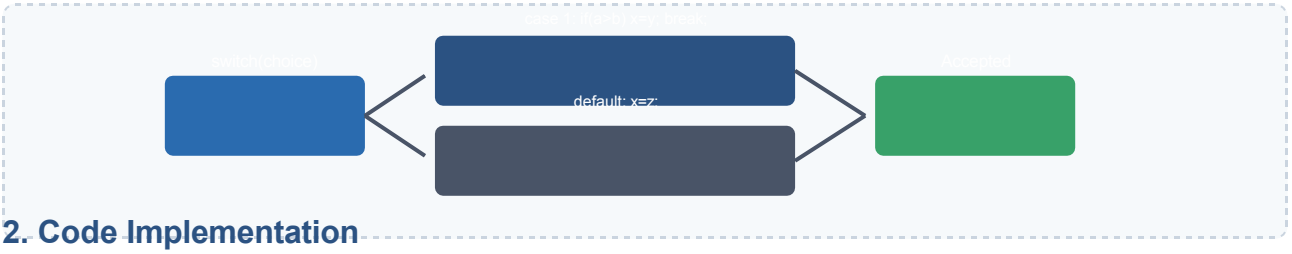
QUESTION 3: Parser for Nested switch–if Statements

Grammar:

- S → switch(id){CaseList} | if(E) id=id; | id=id;
- CaseList → case num : S break; CaseList | default : S
- E → id>id

Sample Input: `switch(choice) { case 1: if(a>b) x=y; break; default: x=z; }`

1. Syntax Flow Diagram



2. Code Implementation

FILE: LEXER3.L

```
%{
#include "y.tab.h"
#include <stdio.h>
#include <stdlib.h>
%}

%option noyywrap

%%

"switch"                { return SWITCH; }
"case"                  { return CASE; }
"default"               { return DEFAULT; }
"break"                 { return BREAK; }
"if"                    { return IF; }
">"                     { return GT; }
"="                     { return ASSIGN; }
"{"                     { return LBRACE; }
"}"                     { return RBRACE; }
"("                     { return LPAREN; }
")"                     { return RPAREN; }
":"                     { return COLON; }
";"                     { return SEMI; }
[0-9]+                  { return NUM; }
[a-zA-Z_][a-zA-Z0-9_]*  { return ID; }
[
]+              { /* ignore whitespace */ }
.                       { return yytext[0]; }

%%
```

FILE: PARSER3.Y

```
%{
#include <stdio.h>
#include <stdlib.h>

extern int yylex(void);
void yyerror(const char *s);
%}

%token SWITCH CASE DEFAULT BREAK IF GT ASSIGN LBRACE RBRACE LPAREN RPAREN COLON SEMI NUM ID

%%

Start : S { printf("Accepted
"); return 0; } ;

S   : SWITCH LPAREN ID RPAREN LBRACE CaseList RBRACE
    | IF LPAREN E RPAREN ID ASSIGN ID SEMI
    | ID ASSIGN ID SEMI
    ;

CaseList
    : CASE NUM COLON S BREAK SEMI CaseList
    | DEFAULT COLON S
    ;

E   : ID GT ID ;

%%

void yyerror(const char *s)
    { printf("Rejected
");
}

int main(void) {
    if (yyparse() == 0) return 0;
    return 1;
}
```

3. Output Verification

```
$ yacc -d parser3.y && lex lexer3.l && gcc y.tab.c lex.yy.c -o parser3
```

```
$ ./parser3 < input3.txt
```

Result: Accepted

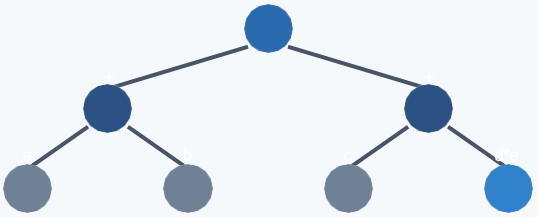
QUESTION 4: Parser for Recursive Arithmetic Expressions

Grammar:

- `E → E + T | T`
- `T → T * F | F`
- `F → id | num | ( E )`

Sample Input: `(a+b)*(c+(d*e))`

1. Parse Tree Structure Diagram



2. Code Implementation

FILE: LEXER4.L

```
%{
#include "y.tab.h"
#include <stdio.h>
#include <stdlib.h>
%}

%option noyywrap

%%

"+"                     { return PLUS; }
"*"                     { return MULT; }
"("                     { return LPAREN; }
")"                     { return RPAREN; }
[0-9]+                  { return NUM; }
[a-zA-Z_][a-zA-Z0-9_]*  { return ID; }
[
]+              { /* ignore whitespace */ }
.                       { return yytext[0]; }

%%
```

FILE: PARSER4.Y

```
%{
#include <stdio.h>
#include <stdlib.h>

extern int yylex(void);
void yyerror(const char *s);
%}

%token PLUS MULT LPAREN RPAREN NUM ID

%%

Start : E { printf("Accepted
"); return 0; } ;

E.  : E PLUS T
    | T
    ;

T   : T MULT F
    | F
    ;

    : ID
    | NUM
    | LPAREN E RPAREN
    ;

%%

void yyerror(const char *s)
    { printf("Rejected
");
}

int main(void) {
    if (yyparse() == 0) return 0;
    return 1;
}
```

3. Output Verification

```
$ yacc -d parser4.y && lex lexer4.l && gcc y.tab.c lex.yy.c -o parser4
```

```
$ ./parser4 < input4.txt
```

Result: Accepted

QUESTION 5: Parser for Array Element Assignments

Grammar:

- `S → id[Index]=E;`
- `Index → num | id`
- `E → id | num`

Sample Input: `marks[i]=90;`

1. Concrete Syntax Structure Diagram

2. Code Implementation

FILE: LEXER5.L

```
%{
#include "y.tab.h"
#include <stdio.h>
#include <stdlib.h>
%}

%option noyywrap

%%

"["                     { return LBRACK; }
"]"                     { return RBRACK; }
"="                     { return ASSIGN; }
";"                     { return SEMI; }
[0-9]+                  { return NUM; }
[a-zA-Z_][a-zA-Z0-9_]*  { return ID; }
[
]+              { /* ignore whitespace */ }
.                       { return yytext[0]; }

%%
```

FILE: PARSER5.Y

```
%{
#include <stdio.h>
#include <stdlib.h>

extern int yylex(void);
void yyerror(const char *s);
%}

%token LBRACK RBRACK ASSIGN SEMI NUM ID

%%

Start : S { printf("Accepted
"); return 0; } ;

S       : ID LBRACK Index RBRACK ASSIGN E SEMI ;

Index   : NUM
        | ID
        ;

E       : ID
        | NUM
        ;

%%

void yyerror(const char *s)
    { printf("Rejected
");
}

int main(void) {
    if (yyparse() == 0) return 0;
    return 1;
}
```

3. Output Verification

```
$ yacc -d parser5.y && lex lexer5.l && gcc y.tab.c lex.yy.c -o parser5
$ ./parser5 < input5.txt
```

Result: Accepted